

文档版本	修订日期	修订说明	作者
V1.0	2026-07-08	结合 WPF .NET 10 现代架构与双人分工重新整理	团队（前端开发 + 后端开发）

目录

- [1. 引言](#)
 - [1.1 项目背景](#)
 - [1.2 项目目标](#)
 - [1.3 项目范围与分工](#)
 - [1.4 术语定义](#)
- [2. 用户需求说明](#)
 - [2.1 用户角色定义](#)
 - [2.2 用户故事](#)
 - [2.3 业务流程图](#)
- [3. 需求分析建模](#)
 - [3.1 用例图](#)
 - [3.2 任务生命周期状态机图](#)
 - [3.3 核心业务交互时序图](#)
 - [3.4 逻辑架构类图 \(MVVM 模式\)](#)
 - [3.5 数据库物理关系图 \(ER 图\)](#)
 - [3.6 数据字典](#)
- [4. View-ViewModel 绑定契约设计 \(双人并联协作核心\)](#)
 - [4.1 MainWindow.xaml \(桌面内嵌看板\) 契约](#)
 - [4.2 SettingsWindow.xaml \(托盘设置中心\) 契约](#)
- [5. 非功能性需求与核心业务规则](#)
 - [5.1 系统开发环境与依赖](#)
 - [5.2 核心业务规则细约约束](#)
 - [5.3 性能与可靠性安全指标](#)

1. 引言

1.1 项目背景

在个人管理与学习认知领域，多数用户常因缺乏规划、遗忘曲线干扰和打卡无及时正反馈而导致学习计划中途流产。传统的打卡软件往往过于厚重，强制弹窗极易打扰日常工作流。开发一个无缝内嵌于 Windows 系统底层桌面、常驻系统托盘、轻量化且具备艾宾浩斯复习提醒的学习养成计划软件，有助于用户建立高效的学习闭环。

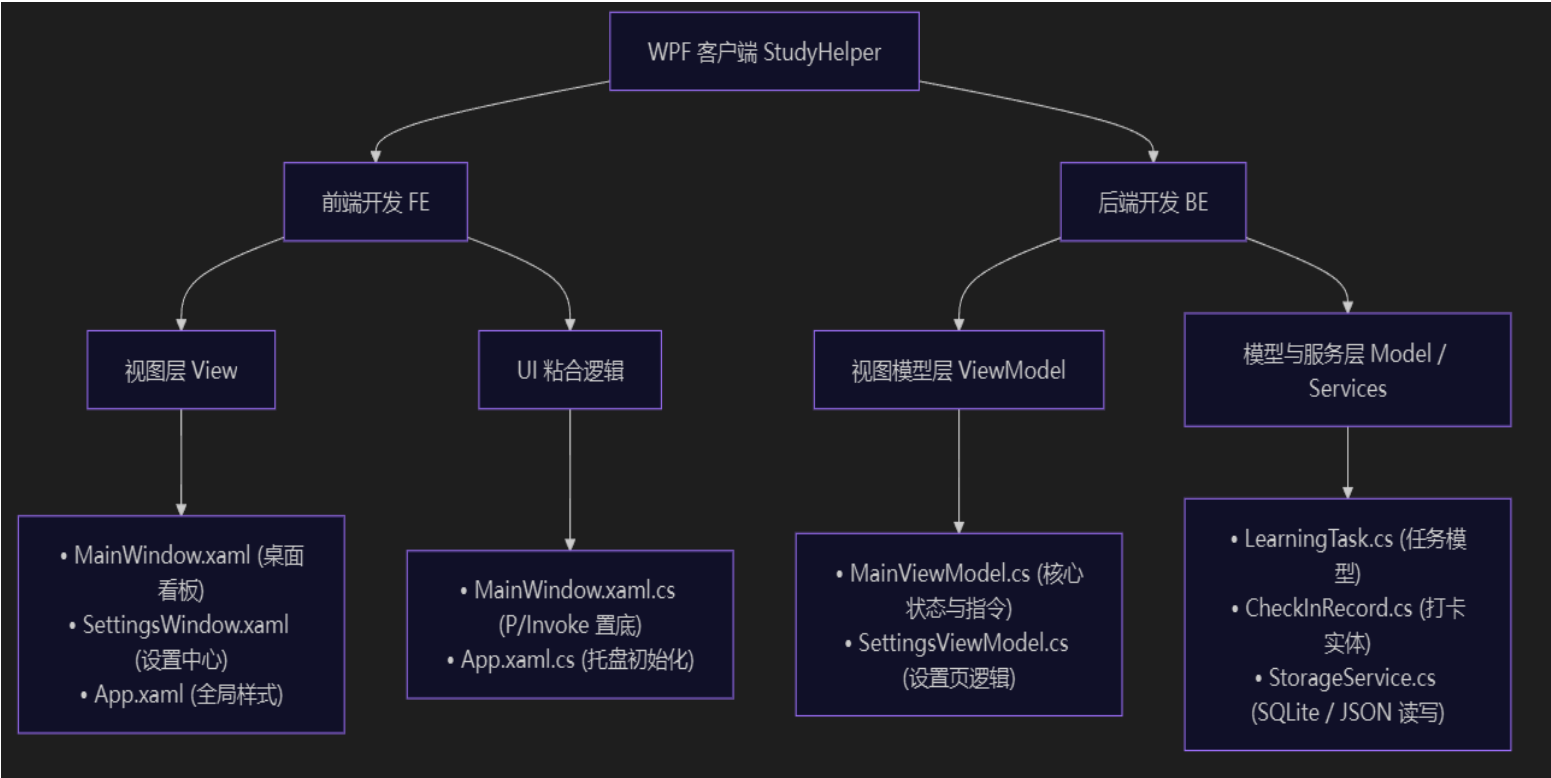
1.2 项目目标

构建一个基于 **C# + WPF (.NET 10.0)** 框架的客户端软件，实现以下核心目标：

- 桌面极简呈现**：提供一个无缝嵌入 Windows 桌面最底层的半透明任务卡片，供用户随时查看今日大任务与 DDL。
- 托盘隐藏与控制**：程序常驻右下角托盘，双击托盘可无缝唤醒全功能设置中心，支持大任务及子任务（小任务）创建、复习设置与 LiveCharts2 动态分析。
- 闭环智能提醒**：应用后台定时器根据优先级、到期时间以及记忆遗忘曲线进行适时弹窗报警，提供打卡和推迟动作。

1.3 项目范围与分工

为实现 2 人开发团队的高效并联开发，项目采用严格的 **MVVM (Model-View-ViewModel)** 软件分工架构。具体模块与物理文件归属如下：



1.4 术语定义

术语	英文	定义
桌面看板	Desktop Widget	无边框、半透明、常驻桌面最底层、不抢占焦点的学习计划展示窗口
设置中心	Settings Panel	托盘双击弹出的配置面板，用于新增任务、查看打卡图表与日历
托盘图标	System Tray Icon	运行在 Windows 任务栏通知区域的隐藏小图标，代表后台守护进程
艾宾浩斯复习	Ebbinghaus Review	根据记忆曲线（第 1, 3, 7, 15 天），由系统在后台自动衍生并提醒复习的功能
P/Invoke	Platform Invoke	C# 显式调用 Windows 原生 C++ Win32 API 的互操作机制

2. 用户需求说明

2.1 用户角色定义

角色	标识	职责说明
软件用户	Student / User	软件的核心操作者。负责在设置中心自定义添加、编辑、下架学习任务，在桌面看板随时监督进度并打卡。
运行系统	OS Environment	Windows 系统环境。负责托管系统的 DispatcherTimer 定时器，托管托盘图标生命周期，处理置底物理重绘。

2.2 用户故事

2.2.1 学习任务模块（Task Module）

- US1.1：作为用户，我想要自定义添加学习大任务，以便规划我个人的长期学习方向。
- US1.2：作为用户，我想要从系统推荐库（如“考研数学打卡”、“背单词 30 天”）一键导入模板任务，以便快速开始。
- US1.3：作为用户，我想要为大任务添加多个层级的子任务（小任务），以便于将复杂的学习目标拆解。
- US1.4：作为用户，我想要为任务设置主题、完成时间（DDL）、优先级（高/中/低），以及是否需要复习提醒，以便于多维度分类和自动归档。

2.2.2 任务提醒模块（Reminder Module）

- US2.1：作为用户，我想要系统根据设定的到期完成时间发送通知，以免遗漏重要的 Deadline。
- US2.2：作为用户，我希望高优先级的任务能比低优先级任务获得更频繁的提醒，以便我能将精力倾斜于紧急事项。

- **US2.3:** 作为用户，我想要开启“复习提醒”后，系统能在我完成打卡后的第 1、3、7、15 天在后台自动衍生复习子任务，以对抗遗忘。

2.2.3 任务打卡模块 (Check-in Module)

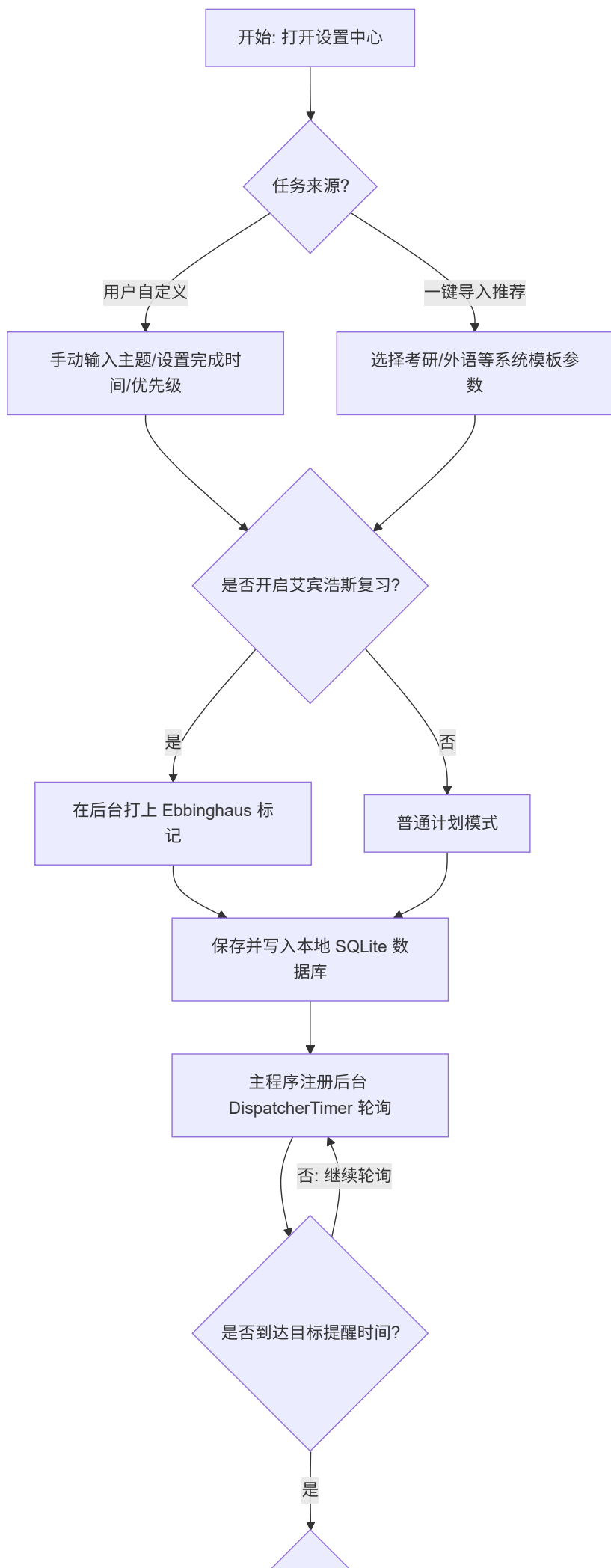
- **US3.1:** 作为用户，我想要在完成学习后在桌面一键“打卡”，以便系统能精确记录我每天的学习轨迹。
- **US3.2:** 作为用户，我希望系统能根据我每天的打卡动作，自动更新并累计连续打卡天数，以此提供正反馈。

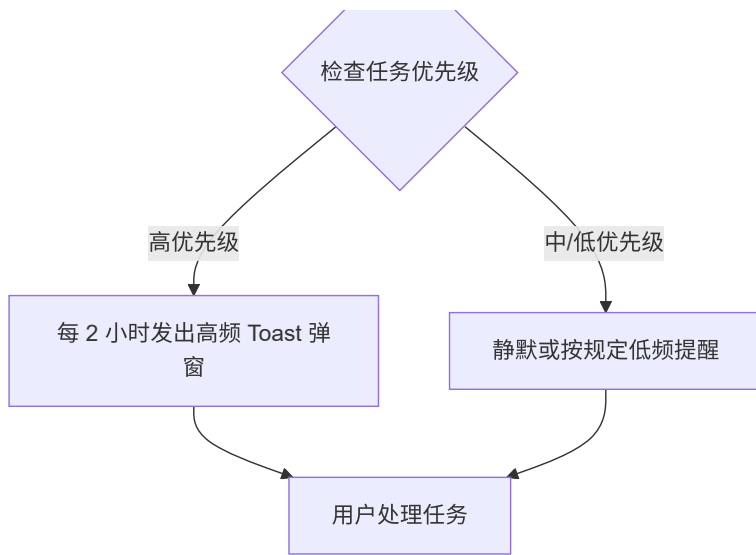
2.2.4 可视化数据分析模块 (Analytics Module)

- **US4.1:** 作为用户，我想要通过界面上的圆形进度条和柱状图，直观查看我每天的任务添加数和打卡完成率。
 - **US4.2:** 作为用户，我想要通过日历卡片，回溯并高亮显示过去一个月中我的具体打卡完成日期，以便于自省。
-

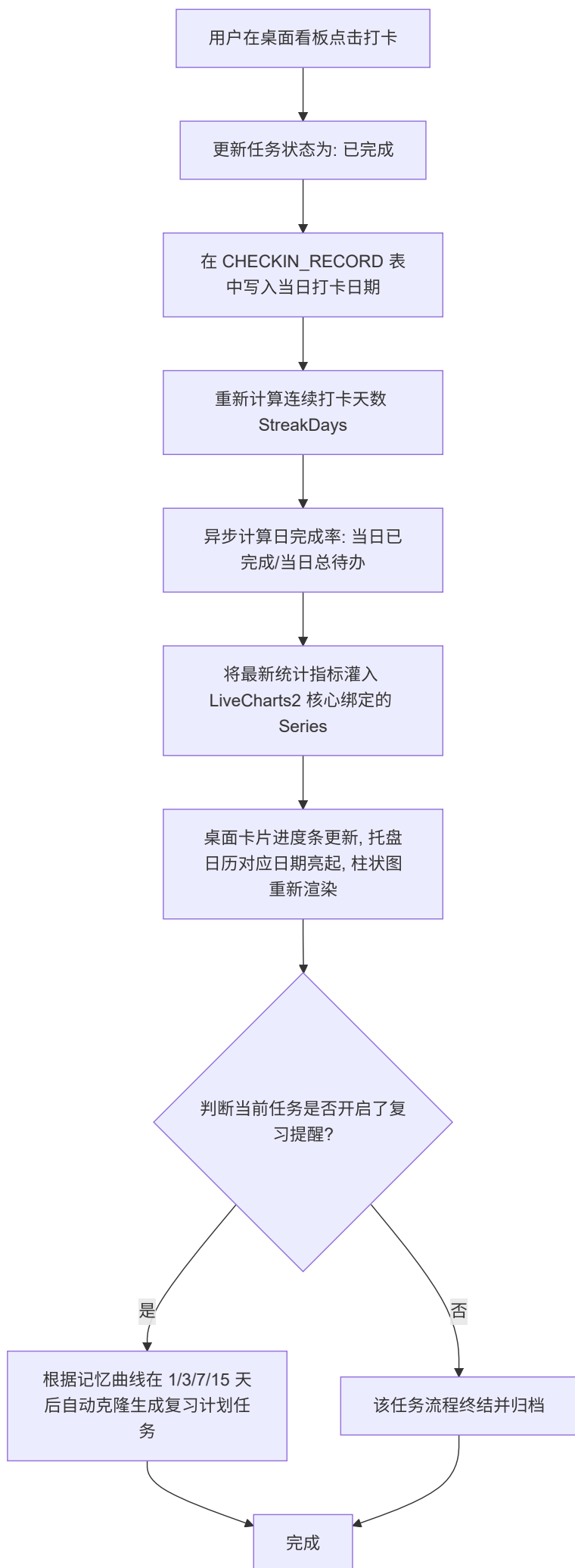
2.3 业务流程图

2.3.1 任务创建、推荐与提醒业务流程



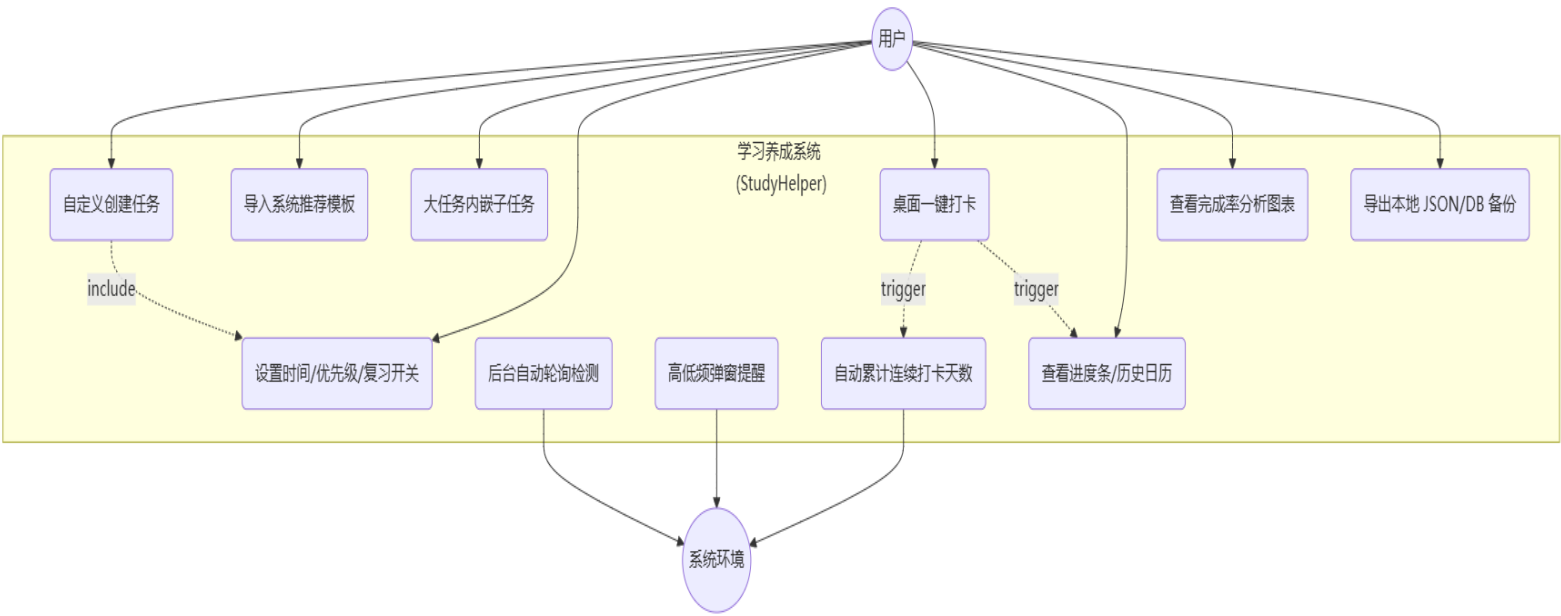


2.3.2 任务打卡、连续天数与数据分析图表流程

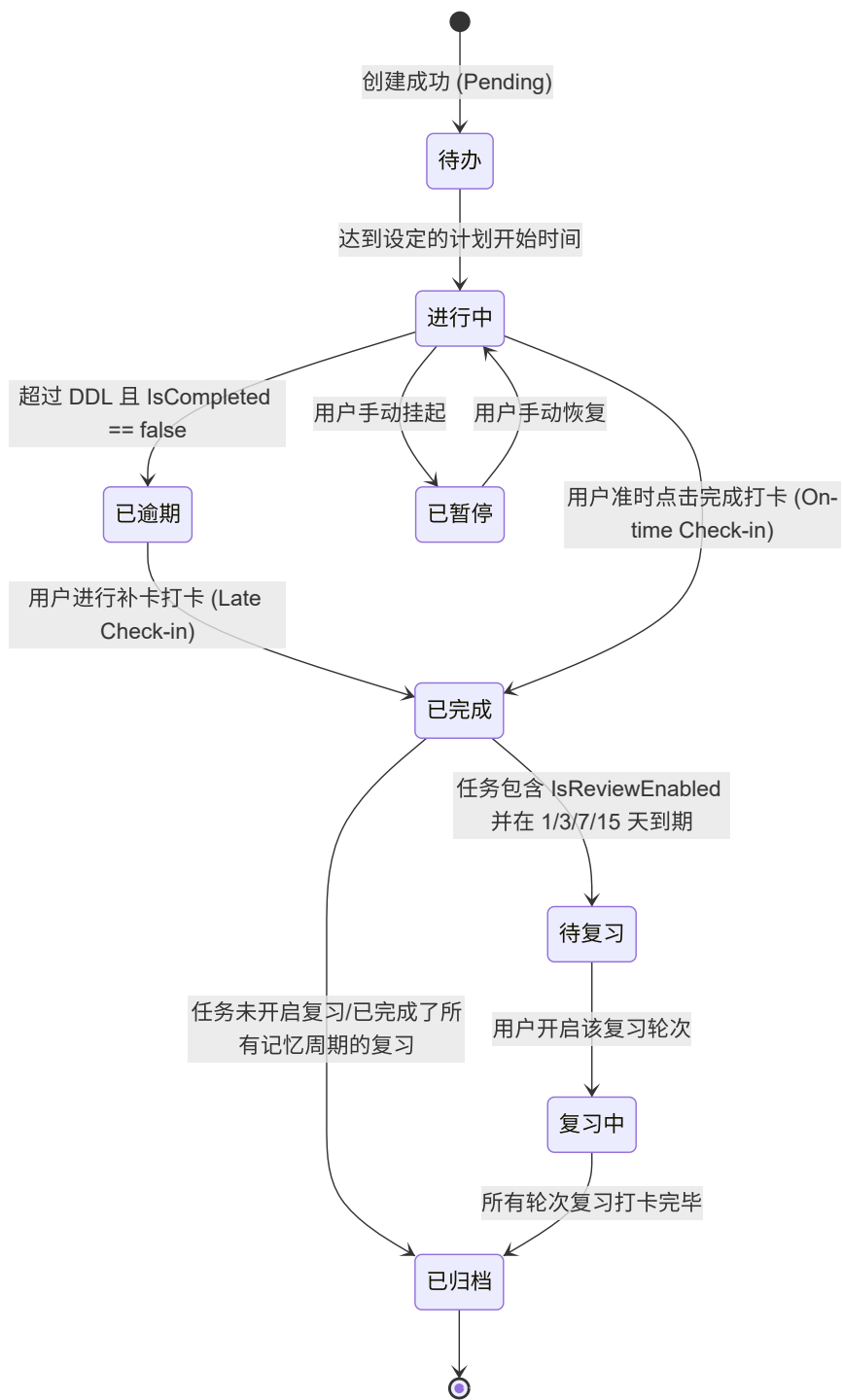


3. 需求分析建模

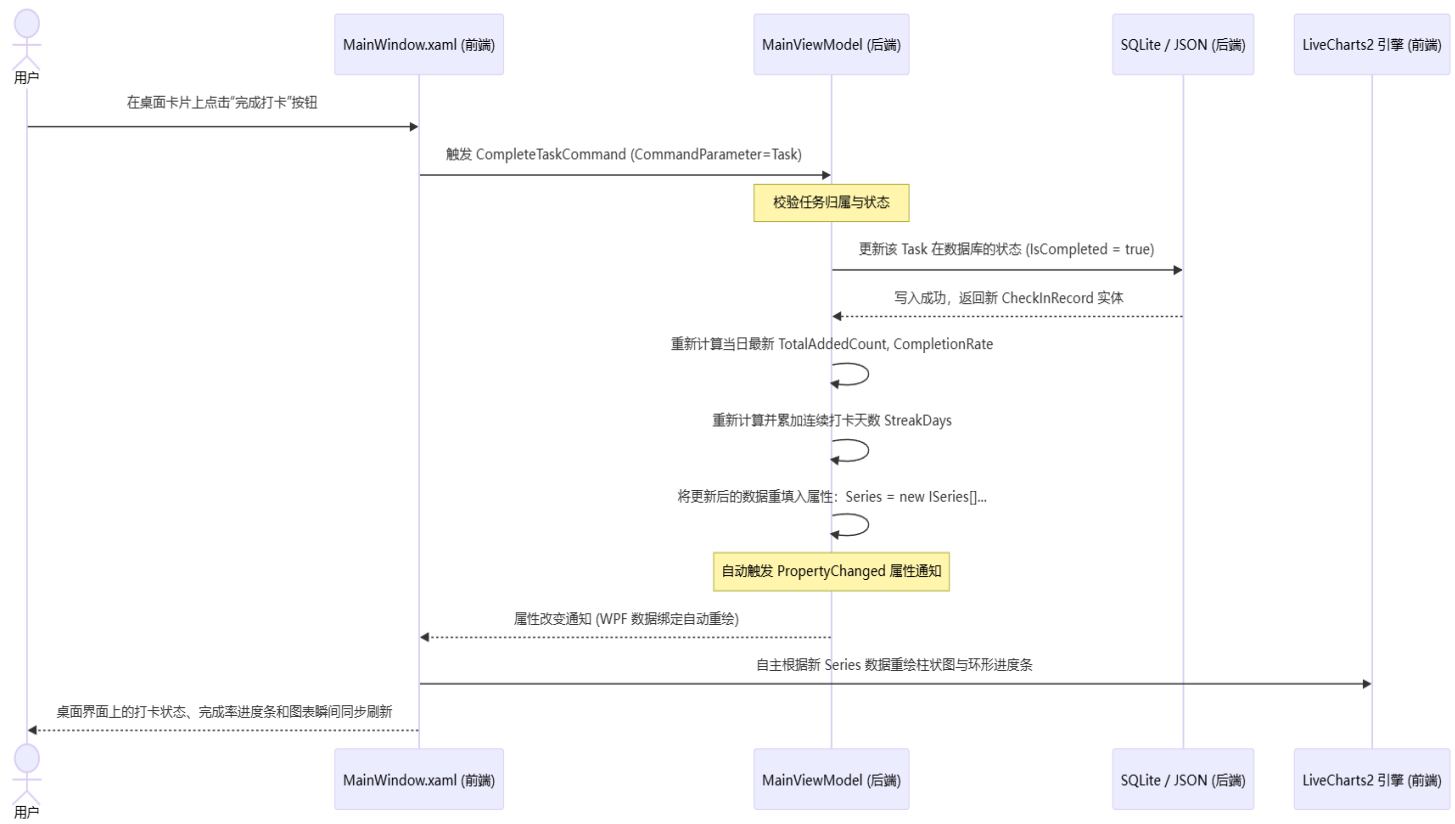
3.1 用例图



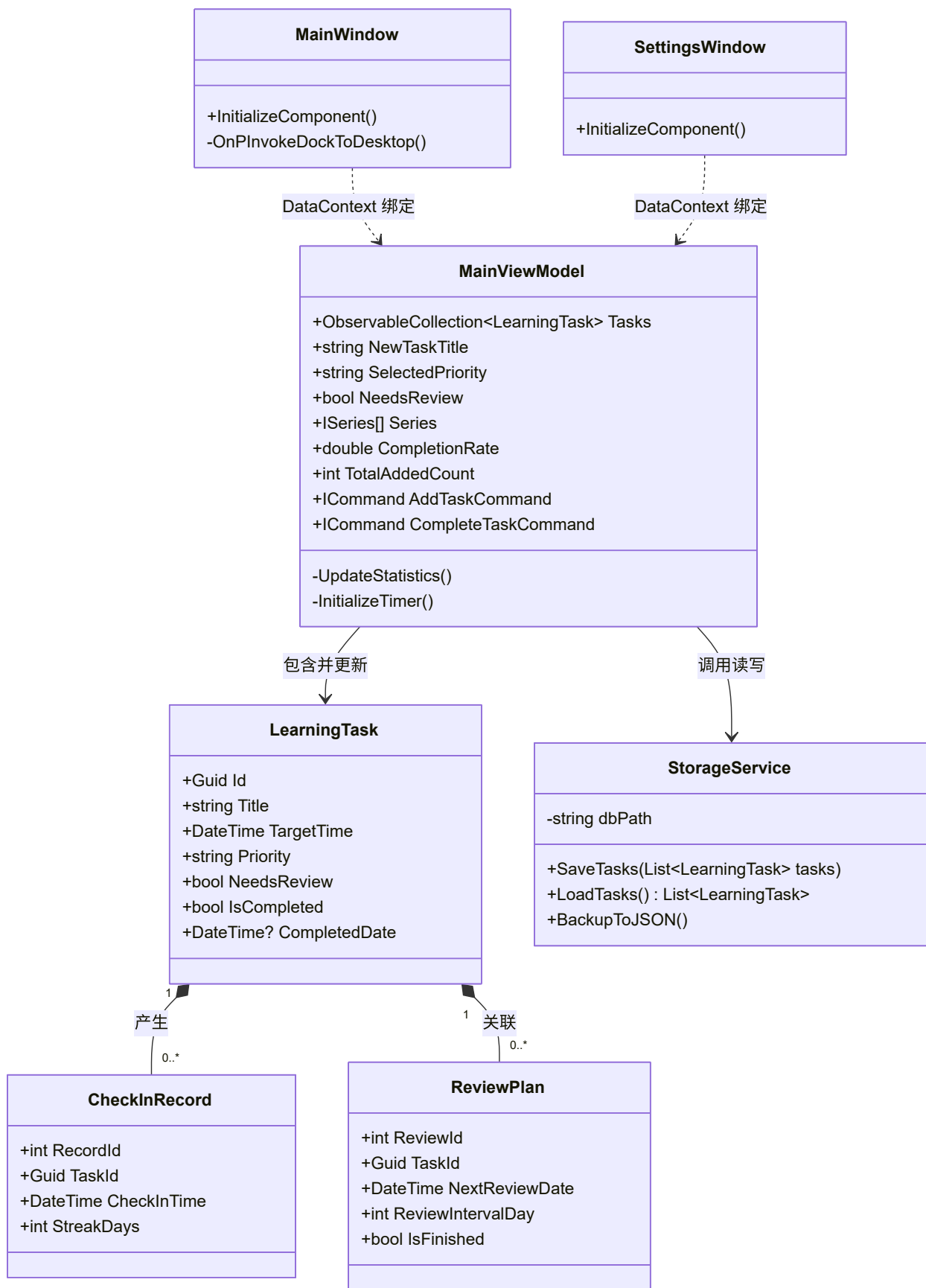
3.2 任务生命周期状态机图



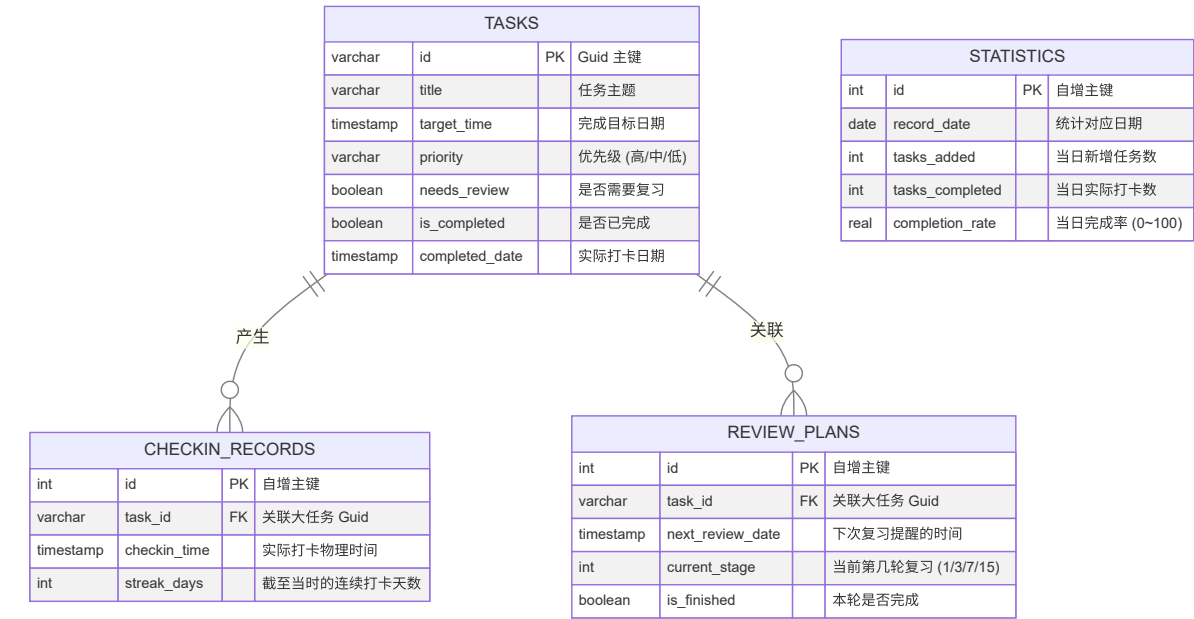
3.3 核心业务交互时序图（打卡及 LiveCharts2 刷新）



3.4 逻辑架构类图（MVVM 模式与分工）



3.5 数据库物理关系图 (ER 图 - 后端开发负责)



3.6 数据字典

3.6.1 任务物理表 (TASKS)

字段名	物理数据类型	长度	约束	默认值	说明
id	VARCHAR	36	PRIMARY KEY	-	本地通过 Guid 自动生成的 36 位唯一码
title	VARCHAR	255	NOT NULL	-	学习大任务的主题名称
target_time	TIMESTAMP	-	NOT NULL	-	设定的截止日期 (DDL)
priority	VARCHAR	10	CHECK IN ('高','中','低')	'中'	优先级，会影响到期提醒的轮询频率
needs_review	BOOLEAN	-	NOT NULL	FALSE	是否在该任务打卡后，依据艾宾浩斯曲线派生复习任务
is_completed	BOOLEAN	-	NOT NULL	FALSE	标识大任务是否已经完成并提交了打卡
completed_date	TIMESTAMP	-	NULL	NULL	实际发生打卡动作的物理时刻

3.6.2 实际打卡记录表 (CHECKIN_RECORDS)

字段名	物理数据类型	长度	约束	默认值	说明
id	INTEGER	-	PRIMARY KEY AUTOINCREMENT	-	打卡记录唯一自增 ID
task_id	VARCHAR	36	FOREIGN KEY -> TASKS(id)	-	发生打卡的大任务 ID
checkin_time	TIMESTAMP	-	NOT NULL	CURRENT_TIMESTAMP	手动点击打卡的时间戳
streak_days	INTEGER	-	NOT NULL	1	截至当前该记录连续坚持的天数

4. View-ViewModel 绑定契约设计（双人并联协作核心）

为达成前端开发者（只写 XAML）与后端开发者（只写 C# 逻辑）**绝不干涉彼此物理文件**的目标，双方必须严格以此绑定契约为物理交界线进行独立作业：

4.1 MainWindow.xaml（桌面内嵌卡片看板）契约

前端在 `MainWindow.xaml` 中只需声明以下 `{Binding ...}`，后端在 `MainViewModel.cs` 中必须确保命名与数据类型一致：

XAML 控件类型	绑定的 XAML 属性	契约绑定属性名（VM 对应）	数据类型	用途/说明
Listbox	ItemsSource	Tasks	ObservableCollection<LearningTask>	自动绑定大任务列表，前端通过 DataTemplate 显示每个任务的 DDL 与优先级
Button (Listbox 内部)	Command	DataContext.CompleteTaskCommand	ICommand	点击“完成打卡”时触发。前端必须将 CommandParameter 绑定为 {Binding}（即当前项）
Button (顶部×)	Command	ExitApplicationCommand	ICommand	彻底关闭应用，释放系统托盘、定时器，终止后台进程并关闭窗口
TextBlock	Text	StreakDays	int	显示用户当前累计的“连续打卡天数”

4.2 SettingsWindow.xaml（托盘设置中心）契约

后端在后台初始化 LiveCharts 或计算百分比，前端只管设计各种控件的外观排版与色调：

XAML 控件类型	绑定的 XAML 属性	契约绑定属性名（VM 对应）	数据类型	用途/说明
TextBox	Text	NewTaskTitle	string (UpdateSourceTrigger=PropertyChanged)	输入框中新任务的名称
DatePicker	SelectedDate	SelectedDate	DateTime	自定义任务的 Deadline
ComboBox	ItemsSource SelectedItem	Priorities SelectedPriority	ObservableCollection<string> string	优先级选择器（高、中、低）
CheckBox	IsChecked	NeedsReview	bool	勾选此项则在打卡后自动启动 1/3/7/15 周期提醒
Button (添加)	Command	AddTaskCommand	ICommand	触发在后端列表中新增一条数据记录，并清空输入框
ProgressBar	Value	CompletionRate	double	动态绑定今日打卡完成率（0%~100%）
CartesianChart	Series	Series	LiveChartsCore.ISeries[]	LiveCharts2 柱状图/折线图专用绑定属性，后台更新后图表可自动重画
Calendar	SelectedDates	HistoryCheckInDates	SelectedDatesCollection（或后端代理绑定）	在托盘日历中高亮渲染有打卡记录的特定日期

5. 非功能性需求与核心业务规则

5.1 系统开发环境与依赖

维度	规范与技术选型	说明
前端/UI 开发	WPF (Windows Presentation Foundation)	采用最新 .NET 10.0 API，不再使用落后的旧版 .NET Framework。
架构模式	MVVM (Model-View-ViewModel)	引入微软官方维护的 CommunityToolkit.Mvvm 包。

维度	规范与技术选型	说明
图表可视化	LiveChartsCore.SkiaSharpView.WPF	前后端基于跨平台的 SkiaSharp 矢量引擎绘制统计卡片。
托盘图标实现	System.Windows.Forms.NotifyIcon	在 .csproj 中设置 <UseWindowsForms>true</UseWindowsForms>，不引入第三方不稳定的 NotifyIcon 包装，直接调用 Windows 系统原生托盘并安全退出。

5.2 核心业务规则细约束

1. 提醒频率规则（通过后台定时轮询检测）

- **高优先级任务**：截止目标日期前 3 天，每天弹出 Toast 提醒 2 次；截止日期当天，每隔 2 小时弹出 1 次。
- **中优先级任务**：截止目标日期前 1 天，当天提醒 1 次；截止日期当天，共弹出提醒 2 次。
- **低优先级任务**：仅在截止日期当天提醒 1 次。

2. 复习提醒规则（艾宾浩斯记忆曲线简化版）

- 在带有 `IsReviewEnabled == true` 属性的任务被用户打卡（`IsCompleted` 变为 `true`）后，系统会启动拦截逻辑。
- 系统将自动往 `REVIEW_PLANS` 物理表中写入 4 个复习时间节点：
 - **复习一阶段**：打卡后第 1 天；
 - **复习二阶段**：打卡后第 3 天；
 - **复习三阶段**：打卡后第 7 天；
 - **复习四阶段**：打卡后第 15 天。
- 当到达对应阶段的日期时，后台定时器会将该复习计划唤醒，重新生成复习提醒，并在设置面板的列表中显示为“待复习任务”。

3. 完成率计算公式

$$\text{日完成率} = \frac{\text{当日已打卡的任务数}}{\text{当日处于“待办/进行中/已逾期”的任务总数}} \times 100\%$$

4. 打卡物理约束

- **准时打卡**：只能在任务设置的 DDL 截止当天 24:00 前完成打卡，计入连续打卡天数（`StreakDays`）的累加。
- **逾期断签**：一旦超过 DDL 的次日 0:00 仍未打卡，系统会将任务强制标为“已逾期”，此时连续打卡天数（`StreakDays`）立即清零归 1，打卡日历中不会高亮对应日期。

5.3 性能与可靠性安全指标

- **冷启动速度**：桌面内嵌卡片及系统托盘在双击 `.exe` 后展示耗时 ≤ 1.5 秒。
- **物理内嵌置底**：前端调用 `SetWindowPos` 必须拦截所有 Windows 系统快捷键（如 Win+D）对该看板的影响，使其永久死死贴在壁纸上方、不抢占用户当前输入焦点。
- **本地数据安全性**：所有的 `tasks.json` 备份文件或 SQLite 本地数据文件，必须存储在 Windows 的用户隐藏目录 `C:\Users\<Username>\AppData\Local\StudyHelper` 中，防止用户误删数据。